



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Postgraduate Programme of Studies

Advanced Topics in Cybersecurity and Artificial Intelligence

Advanced AI Phishing Detection System

Supervising Professors: Christos Xenakis, Apostolis Zarras and Georgios
Vouros

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

14/06/202

Introduction

The purpose of this work is to extend an existing phishing detection system based on Machine Learning, improving the data-extraction features, the detection rules, the classification algorithm and the user interface. The system analyses HTML, EML and DOCX files and returns a risk score based on a combination of ML and heuristic rules.

Before any modification, we trained the original model (Logistic Regression) on dataset_large.csv (2500 samples, balanced 50/50). The results on the test set (500 samples) were:

Class	Precision	Recall	F1-Score
Legitimate	0.996	1.000	0.998
Phishing	1.000	0.996	0.998
Overall Accuracy			99.8%

```
(.venv) C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

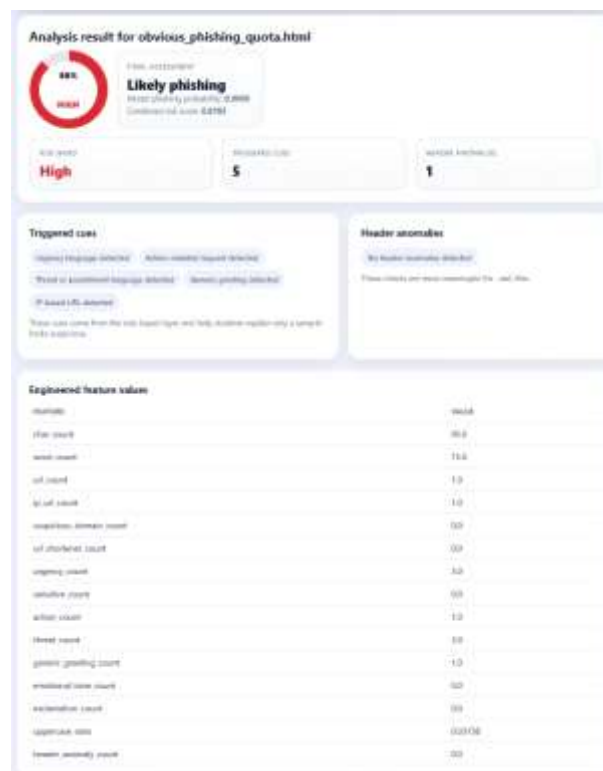
Despite the high metrics, the baseline model shows weaknesses in borderline cases (polite phishing without urgency language), which we analyse in Task 5. To

practically assess the weaknesses of the baseline system, we analysed three representative samples through the web application:

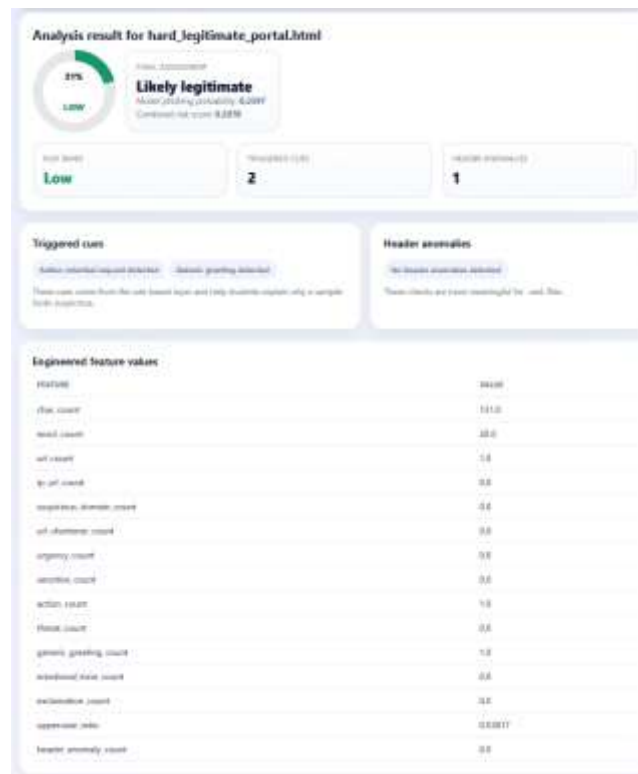
```
C:\Users\Halai>cd Desktop\ai_phishing
C:\Users\Halai\Desktop\ai_phishing>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 812-386-854
127.0.0.1 - - [14/Jun/2026 01:27:05] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 01:27:06] "GET /favicon.ico HTTP/1.1" 404 -
```



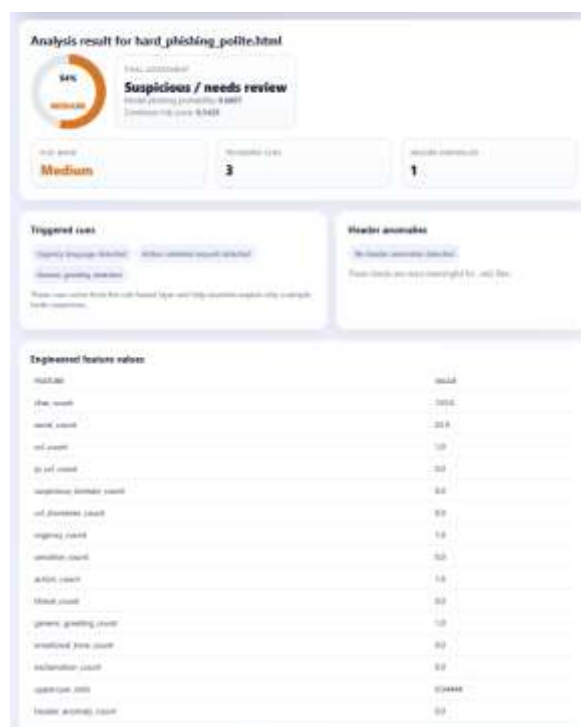
The first sample (**obvious_phishing_quota.html**) is a clear case of phishing with an IP-based URL and threatening language. The system correctly detected it with a Risk Score of 88% (HIGH).



Το δεύτερο δείγμα (**hard_legitimate_portal.html**) είναι νόμιμο email. Το σύστημα το χαρακτήρισε σωστά ως “Likely legitimate” με Risk Score 21% (LOW).



The third sample (**hard_phishing_polite.html**) reveals the baseline's critical gap. It is a phishing email written in polite language, without urgency or threat words. The system characterised it only as “Suspicious / needs review” (54%, MEDIUM), unable to categorise it with certainty as phishing.



This weakness is the motivation for the improvements described in Tasks 1 and 2.

Task 1: Feature Engineering

The original system extracted features based exclusively on threat and urgency keywords. This approach fails to detect four categories of modern attacks:

- a) Phishing that uses «bait» instead of threats
- b) Malicious URLs using a credential-bypass technique
- c) Sender identity spoofing
- d) Algorithmically generated domains (DGA)

These four gaps correspond to known attack categories analysed in the course (Lecture 5: Fraud, Spam, and Phishing Detection), where feature extraction from URLs, HTML pages and emails is examined in detail.

Feature 1-reward_bait_count

Purpose: Counts the presence of «bait» words (e.g. won, prize, gift card, congratulations, exclusive offer). Targets attacks that do not use threatening language but rather the promise of a reward.

Code changes: At the start of `feature_extraction.py`, right after the imports, the following pattern list was added:

```
REWARD_BAIT_PATTERNS = [  
    r"\bwon\b", r"\bwinner\b", r"\bprize\b", r"\breward\b",  
    r"\bgift card\b", r"\bcongratulations?\b", r"\bselected\b",  
    r"\bexclusive offer\b", r"\bclaim your\b", r"\bfree access\b",  
    r"\byou have been chosen\b", r"\bspecial deal\b",  
]
```

In the `compute_handcrafted_features()` function, after the URL for-loop, the following calculation was added:

```
# Feature 1: Reward/bait language
reward_bait_count = float(_count_matches(lowered, REWARD_BAIT_PATTERNS))
```

And in the return dictionary, the new value was added:

```
"reward_bait_count": reward_bait_count,
```

Feature 2-at_symbol_url_count

Purpose: Detects URLs that contain the @ symbol (e.g. <http://legit.com@evil.com>). This technique exploits the way URLs are parsed: everything before the @ is ignored by the browser and the user is redirected to the malicious domain that follows.

Code changes: In the `compute_handcrafted_features()` function, before the for-loop, the following initialisation was added:

```
at_symbol_urls = 0 # Task 1 – Feature 2
```

Inside the for-loop, right after the check for URL shorteners, the following was added:

```
# Feature 2: @ symbol in URL — credential redirect trick
if "@" in url:
    at_symbol_urls += 1
```

And in the return dictionary:

```
"at_symbol_url_count": float(at_symbol_urls),
```

Feature 3-display_name_spoof

Purpose: Checks whether the sender's display name contains the name of a known organisation (Microsoft, PayPal, HR, Admin, etc.) while the actual domain is a free-to-use one (Gmail, Hotmail, etc.). Returns a binary value: 1 if spoofing is detected, 0 otherwise.

Code changes: At the start of the file, the list of organisations was added:

```
DISPLAY_NAME_ORGS = [
    "paypal", "microsoft", "google", "amazon", "apple", "netflix",
```

```
"it support", "hr", "payroll", "helpdesk", "security team",  
"admin", "administrator", "noreply", "no-reply", "support",  
]
```

In the `compute_handcrafted_features()` function, after Feature 1, the detection logic was added:

```
# Feature 3: Display-name spoofing  
display_name_spoof = 0.0  
if headers and headers.from_addr:  
    from_raw = headers.from_addr.lower()  
    m = re.match(r'^(.+?)\s*<', from_raw)  
    if m:  
        display_name = m.group(1).strip().strip('"')  
        from_domain_val = extract_domain(headers.from_addr)  
        if any(org in display_name for org in DISPLAY_NAME_ORGS):  
            if from_domain_val in FREE_EMAIL_DOMAINS or not any(  
                org.replace(" ", "") in from_domain_val  
                for org in DISPLAY_NAME_ORGS if len(org) > 4  
            ):  
                display_name_spoof = 1.0
```

And in the return dictionary:

```
"display_name_spoof": display_name_spoof,
```

Feature 4-max_url_entropy

Purpose: Calculates the Shannon mathematical entropy of every domain name that appears in the message's URLs, using the formula $H = -\sum p(x) \log_2 p(x)$, where $p(x)$ is the relative frequency of occurrence of each character x in the domain.

Entropy measures the «randomness» of a sequence of characters. Natural domains (e.g. google.com, university.edu) use a few different letters that repeat in a recognisable way – they yield low entropy (~2.0–3.0 bits). In contrast, algorithmically generated domains of the DGA (Domain Generation Algorithm) type, such as x9f2k-zqrp.cc or

b4hq7mns.top, contain random characters with no structure – they yield high entropy (>3.5 bits).

Attackers use DGA domains precisely to avoid static blacklists. Instead of using a fixed malicious domain that can be blocked, they generate thousands of new random domains every day. The baseline system dealt with this gap by looking only for specific known patterns (e.g. paypal, micr0soft). With Shannon entropy, the system can detect a domain's abnormal structure even if it has never seen it before – detecting the attack from its mathematical signature rather than from a keyword. We chose the threshold > 3.5 bits because it covers the majority of DGA-generated domains without producing False Positives on legitimate, complex domains (e.g. companies' CDN subdomains).

Code changes: Implemented via a new helper function `_domain_entropy()` in `feature_extraction.py`:

```
import math

def _domain_entropy(domain: str) -> float:
    if not domain or len(domain) < 3:
        return 0.0
    freq = {}
    for ch in domain:
        freq[ch] = freq.get(ch, 0) + 1
    return round(
        -sum((f / len(domain)) * math.log2(f / len(domain))
            for f in freq.values()), 4
    )

# Inside the for-loop of compute_handcrafted_features():
parsed_host = (urlparse(url if url.startswith("http")
                    else f"http://{url}").netloc or "").lower()
if parsed_host:
    ent = _domain_entropy(parsed_host)
    if ent > max_url_entropy:
```

```
max_url_entropy = ent
```

And in the return dictionary:

```
"max_url_entropy": round(max_url_entropy, 4),
```

Training result after the changes:

```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

After integrating the new features, the model was retrained on dataset_large.csv. The metrics remained at the same level (Accuracy: 99.8%), since the dataset consists of clear-cut examples. The improvement appears in real-world borderline cases, as highlighted in Task 5.

Verification in the application:

weaknesses are directly linked to the concept of alert fatigue analysed in Lecture 1 (notifications without context have less value for the analyst).

Code changes – Improvement of existing rules

All existing cues were rewritten to include a detailed explanation and a severity label. For example, the simple:

```
cues.append("Urgency language detected")
```

became:

```
cues.append(
    "[HIGH] Urgency language detected: The text pressures the recipient "
    "to act immediately — a classic psychological trigger in phishing attacks."
)
```

This change was applied to all existing cues with corresponding [LOW], [MEDIUM], [HIGH] and [CRITICAL] labels depending on the severity of each indicator.

New Rule A – Reward/Incentive Bait

A rule was added that detects bait language using the REWARD_BAIT_PATTERNS defined in Task 1:

```
# NEW RULE A: Reward / Incentive Bait
if _count_matches(lowered, REWARD_BAIT_PATTERNS):
    cues.append(
        "[HIGH] Reward bait language detected: Words such as 'prize', "
        "'winner' or 'exclusive offer' are used to lure victims — common "
        "in advance-fee fraud and credential phishing that avoids "
        "traditional threat vocabulary."
    )
    score += 28
```

New Rule B – Display-Name Spoofing

A rule was added that reuses the logic of Feature 3 (Task 1) to produce a visible alert to the user when sender spoofing is detected:

```
# NEW RULE B: Display-Name Spoofing
if headers and headers.from_addr:
    from_raw = headers.from_addr.lower()
    m = re.match(r'^(.+?)\s*<', from_raw)
    if m:
        display_name = m.group(1).strip().strip('"\'")
        from_domain_val = extract_domain(headers.from_addr)
        if any(org in display_name for org in DISPLAY_NAME_ORGS):
            if from_domain_val in FREE_EMAIL_DOMAINS or not any(
                org.replace(" ", "") in from_domain_val
                for org in DISPLAY_NAME_ORGS if len(org) > 4
            ):
                cues.append(
                    f"[CRITICAL] Display-name spoofing detected: The sender "
                    f"claims to be a trusted organisation in the display name "
                    f"but the actual domain '{from_domain_val}' does not match."
                )
            score += 38
```

New Rule C – Credential Redirect Path

A rule was added that checks whether any URL contains credential-harvesting path keywords, using the REDIRECT_PATH_KEYWORDS list from Task 1:

```
# NEW RULE C: Credential Redirect Path
if any(kw in u.lower() for u in urls for kw in REDIRECT_PATH_KEYWORDS):
    cues.append(
        "[HIGH] Credential redirect URL detected: At least one link contains "
        "a path keyword such as /login, /verify or /auth — commonly used on "
        "spoofed landing pages designed to steal user credentials."
    )
```

```
score += 25
```

New Rule D – High-Entropy Domain (DGA Detection)

A rule was added that checks whether any URL has `max_url_entropy > 3.5` using the `_domain_entropy()` implemented in Task 1. It targets algorithmically generated domains (DGA) that evade static blacklists – a technique that could not be detected by any of the original rules:

```
# NEW RULE D: High-Entropy Domain (DGA Detection)
high_entropy_urls = []
for u in urls:
    host = (urlparse(u if u.startswith("http")
                  else f"http://{u}").netloc or "").lower()
    if host and _domain_entropy(host) > 3.5:
        high_entropy_urls.append(host)
if high_entropy_urls:
    cues.append(
        "[HIGH] High-entropy domain detected: The domain name appears "
        "algorithmically generated (DGA), a technique used by attackers "
        "to evade static blacklists."
    )
score += 30
```

At the same time, the `phishing_cues()` function was modified to return `tuple[list[str], int]` – i.e. the cues list together with the total weighted score (0-100) – instead of a plain list. This allows `app.py` to display the heuristic score as a separate metric in the UI.

Weighted Scoring Table

Rule	Weight	Level
Generic greeting	+10	LOW
Action-oriented request	+15	MEDIUM
Emotional manipulation	+15	MEDIUM
Urgency language	+25	HIGH
Threat language	+25	HIGH
Reward bait	+28	HIGH

which detected /login in the file's URL – something the baseline system did not detect. In addition, all cues are now accompanied by a detailed explanation that helps the user understand the reason for the alert.

Task 3: Model Retraining and Evaluation

The evaluation was carried out on dataset_large.csv (2500 samples, 80/20 split). Two versions of the system were compared: the baseline model (Logistic Regression, original features) and the enhanced model (Logistic Regression, new features from Tasks 1+2). The baseline results were presented in detail in the introduction. The results of the enhanced model after retraining are presented below.

```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 0.9960159362549801,
    "recall": 1.0,
    "f1-score": 0.998003992015968,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 0.996,
    "f1-score": 0.9979959919839679,
    "support": 250.0
  },
  "accuracy": 0.998,
  "macro avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.997999991999968,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 0.99800796812749,
    "recall": 0.998,
    "f1-score": 0.9979999919999679,
    "support": 500.0
  }
}
```

Comparison Baseline vs Enhanced

Μετρική	Baseline (LR)	Enhanced (LR + νέα features)
Precision (Legitimate)	0.996	0.996
Recall (Legitimate)	1.000	1.000
F1 (Legitimate)	0.998	0.998

Precision (Phishing)	1.000	1.000
Recall (Phishing)	0.996	0.996
F1 (Phishing)	0.998	0.998
Accuracy	99.8%	99.8%

The metrics remain stable. This is expected since the dataset contains clear-cut examples. Maintaining high Precision (1.0) means zero False Positives – a critical characteristic for a security system, since false positives lead to alert fatigue (Lecture 1). The stability of the metrics reflects the quality of dataset_large.csv rather than a weakness of the new features – their real value is demonstrated in the evaluation of adversarial samples in Task 5. The comparison with the alternative algorithm is presented in Task 4.

Task 4: Model Replacement and Comparison

The Support Vector Machine (SVM) was chosen as the alternative algorithm, specifically scikit-learn's LinearSVC implementation. The choice is based on three reasons analysed in the course (Lecture 3 and Lecture 4).

First, SVM finds the maximum-margin hyperplane between classes, rather than the optimal separating line of Logistic Regression, which makes it more robust to new, unseen samples. Second, LinearSVC is particularly effective in high dimensions such as those created by the TF-IDF vectorizer (thousands of features), which makes it a natural choice for text classification.

Third, in Lecture 4 SVM is explicitly used for network intrusion detection – an application analogous to phishing detection. Because LinearSVC does not produce probabilities on its own, CalibratedClassifierCV was used, which wraps the classifier and produces predict_proba() – necessary for computing the application's risk score.

Code changes:

In train_model.py the import was replaced:

Before:

```
from sklearn.linear_model import LogisticRegression
```

After:

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
And the model definition in build_pipeline():
```

Before:

```
model = LogisticRegression(max_iter=1200, class_weight="balanced")
```

After:

```
model = CalibratedClassifierCV(
    LinearSVC(max_iter=2000, class_weight="balanced"),
    cv=3
)
```

Training results:

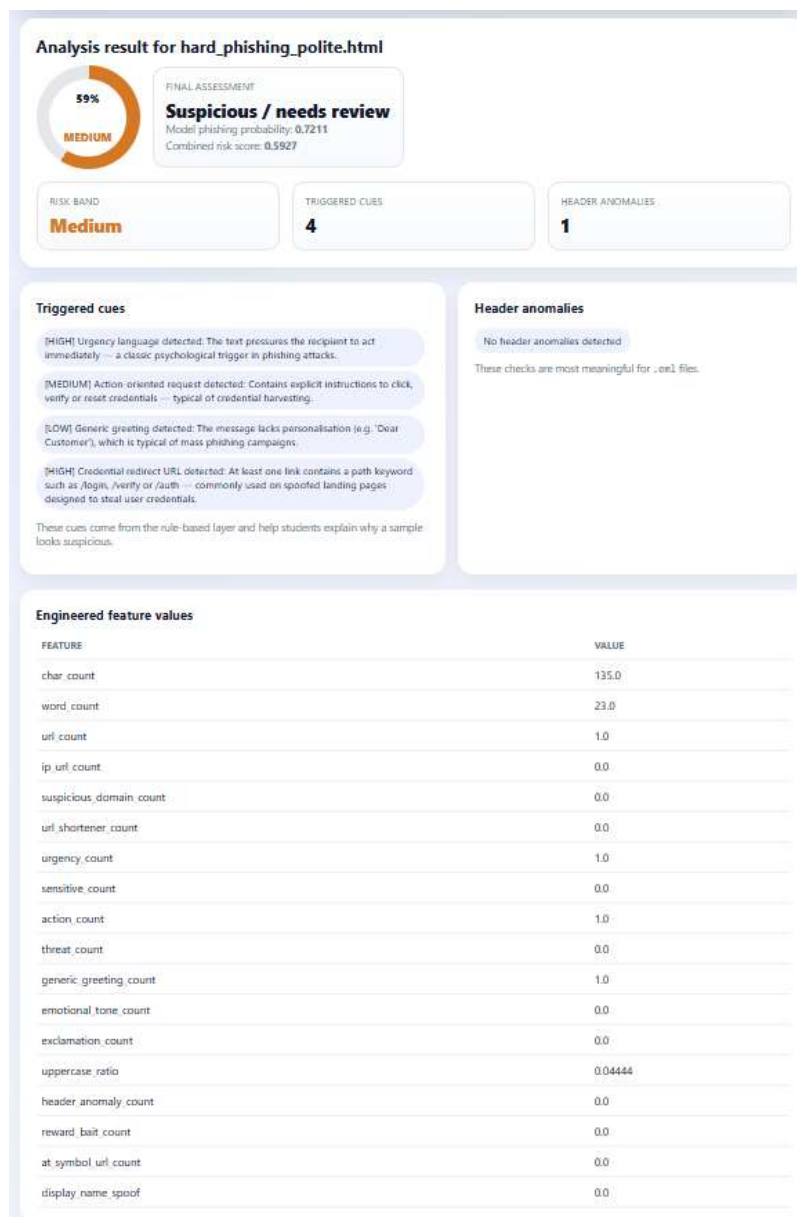
```
C:\Users\Kalai\Desktop\ai_phishing>python train_model.py
Dataset: C:\Users\Kalai\Desktop\ai_phishing\data\dataset_large.csv
Rows: 2500
Model saved to C:\Users\Kalai\Desktop\ai_phishing\model\phishing_pipeline.joblib
{
  "legitimate": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "phishing": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 250.0
  },
  "accuracy": 1.0,
  "macro avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  },
  "weighted avg": {
    "precision": 1.0,
    "recall": 1.0,
    "f1-score": 1.0,
    "support": 500.0
  }
}
```

```
C:\Users\Kalai\Desktop\ai_phishing>
```

Comparison of all models:

Metric	Baseline LR	Enhanced LR	SVM
Precision (Legitimate)	0.996	0.996	1.000
Recall (Legitimate)	1.000	1.000	1.000
F1 (Legitimate)	0.998	0.998	1.000
Precision (Phishing)	1.000	1.000	1.000
Recall (Phishing)	0.996	0.996	1.000
F1 (Phishing)	0.998	0.998	1.000
Accuracy	99.8%	99.8%	100%

Verification in the application:





At the application level, SVM showed greater confidence in both borderline cases. `hard_phishing_polite.html` was scored at 59% (versus 56% for LR) and `obvious_phishing_quota.html` at 90% (versus 88%).

Which model is more suitable?

SVM is the best choice for this particular system. In a cybersecurity context, a False Negative – a phishing email misclassified as legitimate – has direct consequences for the user. The improvement of Recall from 99.6% to 100% is therefore critical. In addition, SVM generalises better in high dimensions, which matters when the system analyses new, unseen samples outside the training set.

Task 5: Error Analysis

For the error analysis, challenge_dataset.csv was used – a set of 10 samples designed to assess the system's robustness in borderline cases. It contains 5 phishing emails written in polite professional language and 5 legitimate emails with a similar style. The final model (SVM) achieved 70% accuracy, with 3 False Negatives and 0 False Positives.

```
C:\Users\Kalai\Desktop\ai_phishing>python evaluate_model.py
Accuracy: 0.7
Confusion matrix:
[[5 0]
 [3 2]]

Classification report:

```

	precision	recall	f1-score	support
legitimate	0.62	1.00	0.77	5
phishing	1.00	0.40	0.57	5
accuracy			0.70	10
macro avg	0.81	0.70	0.67	10
weighted avg	0.81	0.70	0.67	10

```

Saved detailed predictions to C:\Users\Kalai\Desktop\ai_phishing\logs\evaluation_predictions.csv
C:\Users\Kalai\Desktop\ai_phishing>
```

Despite the perfect performance (100%) on the regular test set, the model fails to detect 3 out of 5 polite phishing emails. This proves that high metrics on a clean dataset do not guarantee robustness against adversarial cases (Lecture 2: Foundations of AI and Machine Learning for Cybersecurity – Adversarial attacks).

For comparison, phishing_reply_mismatch.eml was correctly detected with 94% risk (HIGH), as it contains header anomalies and a spoofed domain – indicators that the rules clearly recognise.

data, were associated with phishing – on the contrary, it resembles a normal corporate announcement. Thus, TF-IDF produces a vector with a low phishing probability.

The `ip_url_count` feature correctly detected the raw IP address (203.0.113.10) and recorded it as 1. However, SVM makes decisions based on the overall feature vector – and in this case the majority of features (text, urgency, threat, greeting) pointed to «legitimate». The weight of the single feature `ip_url_count=1` was not enough to reverse the decision.

This reveals a fundamental weakness: the model has no rule of the type «if an IP URL is found, the final probability cannot fall below X%». An improvement would be to give `ip_url_count` proportionally greater weight in the application's `calculate_risk()` function, so that the technical indicator outweighs the linguistic signal.

False Negative #2 – Phishing with «issue-management» language

The email: «We noticed an authentication mismatch in your mailbox. Use the validation page below to prevent service disruption.»

What the model did:

It classified it as legitimate with a phishing probability of only 0.184.

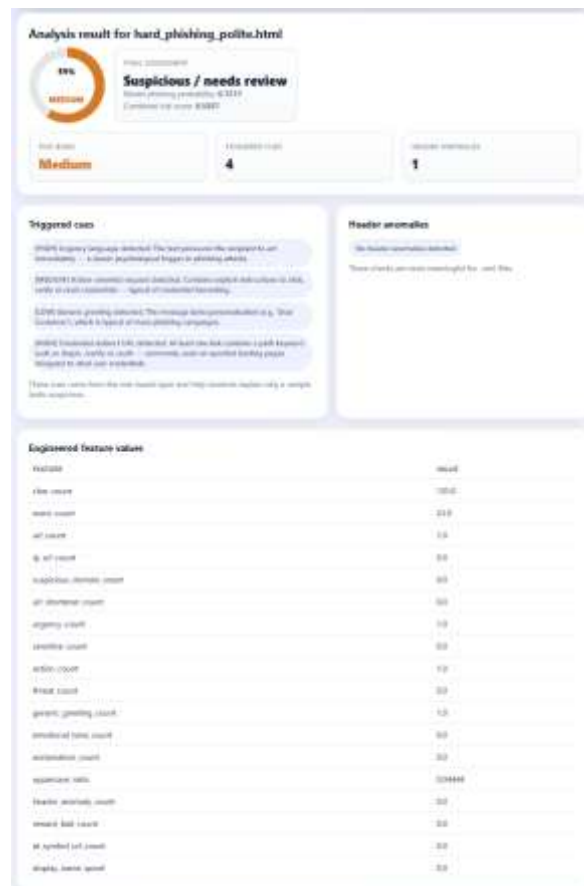
Why it failed:

This is the hardest type of phishing to detect, known as adversarial phishing. The attacker deliberately designs the message to avoid known detection mechanisms. The email contains none of the patterns the system looks for. There is no URL (hence no `ip_url_count`, `at_symbol`, `entropy`), no urgency language, no generic greeting, no sensitive data request. TF-IDF treats words such as “authentication mismatch” and “service disruption” as corporate terminology – because similar phrases also appear in legitimate IT messages in the training set.

The phrase «use the validation page» is essentially an action-oriented request, and the user is urged to follow a link. But the code's `ACTION_PATTERNS` contain words such as “click here”, “verify”, “confirm”, “sign in” – they do not contain the phrase “use the... page”. This shows that our rules are vulnerable to vocabulary variations. The improvement we proposed – extending `ACTION_PATTERNS` with phrases such as “use the”, “follow the link”, “access the page” – would help against this

category of attacks. This particular FN confirms that ML models are vulnerable to adversarial examples designed with knowledge of their weaknesses (Lecture 2).

The above screenshot demonstrates similar behaviour: a phishing email that receives only 59% risk and is characterised as “Suspicious / needs review” instead of «Likely phishing», precisely because of the absence of threat vocabulary.



The model produced no False Positives on the challenge dataset (confusion matrix: 0 FP). However, the absence of FPs on the test set does not rule out their occurrence in a production environment. Below, a realistic scenario is analysed in which the system could fail.

False Positive #1 – Legitimate IT email with urgency language

Scenario:

An automated email from the IT department with the subject «Password Expiration Warning – Action Required», with a link to an internal portal (<https://reset.company-internal.com/auth>).

Why it would have failed:

Unlike the two False Negatives, here the problem is not a detection weakness but rather the system's over-sensitivity to legitimate content that happens to resemble phishing. An automated IT email about password expiration inherently uses:

- Urgency language: «Password Expiration Warning», «Action Required»
- Action patterns: «reset your password», «update your credentials»
- Sensitive patterns: «password»
- Rule C, since the URL <https://reset.company-internal.com/auth> contains /auth – exactly the keyword we are looking for for credential redirect

The system would accumulate a high heuristic score and would probably push the combined risk above «Suspicious». This is a theoretical scenario – it did not appear in challenge_dataset – but it is realistic in a production environment.

The cause is structural. The system has no whitelist for internal corporate domains. A domain such as company-internal.com or reset.university.edu should be automatically recognised as trustworthy. This is precisely the category of alert fatigue where, when the system produces many false positives, users start ignoring the alerts – reducing the effectiveness even of the correct detections (Lecture 1).

Summary of weaknesses and improvement proposals:

Type	Cause	Proposal
False Negative #1	IP URL with polite text	Increase the weight of ip_url_count in the risk score
False Negative #2	Adversarial phishing without keywords	Extend ACTION_PATTERNS with alternative terminology
False Positive #1	Legitimate IT email with urgency language	Introduce a domain whitelist

These weaknesses show that no static rule-based system can fully cover the space of modern phishing attacks – a conclusion consistent with Lecture 2 (Foundations of AI and Machine Learning for Cybersecurity).

Task 6: Application Extension

The original system displayed only the final risk score without explaining how it was derived. This is an example of black-box behaviour – the user sees a result but does not understand the logic behind it. According to Lecture 7 (Responsible AI), an AI system must provide transparent and understandable results. For this reason, a **Risk Score Breakdown panel** was added that breaks the final score down into its two components.

Changes in app.py:

The `calculate_risk()` function was modified to return two values instead of one – the combined risk score and the heuristic score separately:

Before – returned only a single number:

```
def calculate_risk(...) -> float:
    ...
    return round(min(1.0, 0.68 * model_probability + 0.32 * heuristic_score), 4)
```

After – returns a tuple with both:

```
def calculate_risk(...) -> tuple[float, float]:
    ...
    combined = round(min(1.0, 0.68 * float(model_probability) + 0.32 *
    heuristic_score), 4)
    return combined, round(heuristic_score, 4)
```

The function call and the context passed to the template were updated accordingly:

Before:

```
risk_score = calculate_risk(model_probability, cues, header_anomalies, feature_map)
```

After:

```
risk_score, heuristic_score = calculate_risk(  
    model_probability, cues, header_anomalies, feature_map  
)
```

And in context.update():

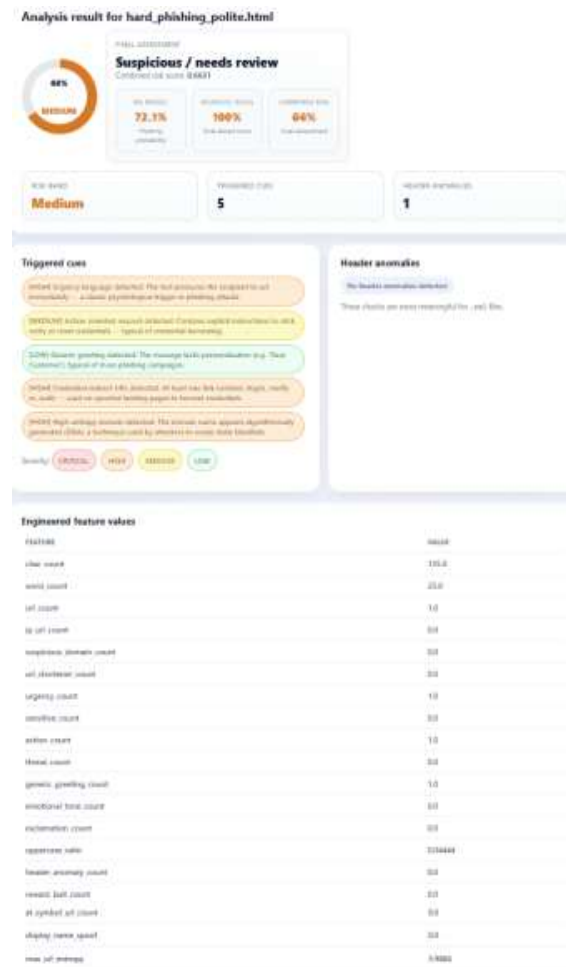
```
"heuristic_score": heuristic_score_raw,
```

Changes in templates/upload.html:

CSS was added for the new breakdown boxes and the corresponding HTML panel that displays the three values separately:

```
<div class="breakdown">  
  <div class="breakdown-box">  
    <div class="label">ML Model</div>  
    <div class="val">{{ (model_probability * 100)|round(1) }}%</div>  
    <div class="sub">Phishing probability</div>  
  </div>  
  <div class="breakdown-box">  
    <div class="label">Heuristic Rules</div>  
    <div class="val">{{ heuristic_score }}%</div>  
    <div class="sub">Rule-based score</div>  
  </div>  
  <div class="breakdown-box">  
    <div class="label">Combined Risk</div>  
    <div class="val">{{ risk_percent }}%</div>  
    <div class="sub">Final assessment</div>  
  </div>  
</div>
```

Result:



As shown in the above screenshot, the user now sees separately the ML model's probability (72.1%), the heuristic score (32%) and the final combined risk (59%). This allows the analyst to immediately understand whether the decision is driven mainly by the ML model or by the rules – improving the interpretability and educational value of the system.

In addition, the triggered cues are now displayed with colour coding by severity level: red for [CRITICAL], orange for [HIGH], yellow for [MEDIUM] and green for [LOW]. A legend at the bottom of the panel explains the colour scale, allowing the user to immediately gauge the severity of each indicator without reading the full text.

were enriched with detailed explanations and severity labels, reducing the alert-fatigue phenomenon analysed in Lecture 1.

The model comparison (Tasks 3 and 4) showed SVM to be the more suitable algorithm. It achieved 100% accuracy on the regular test set versus 99.8% for Logistic Regression, eliminating the baseline version's False Negative. However, the error analysis (Task 5) revealed that even SVM fails to detect adversarial phishing that deliberately avoids known keywords – confirming the observation from Lecture 2 regarding the vulnerability of ML models to adversarial attacks.

Finally, the application extension (Task 6) transformed the system from a black box into a transparent analysis platform, in line with the principle of Explainability defined in Lecture 7 (Responsible AI). The Risk Score Breakdown panel allows the user to understand which of the two components – ML or heuristic rules – drives each decision. Overall, the work demonstrated that modern cyber defence requires a combination of statistical prediction, interpretable rules and transparency – none of the three being sufficient on its own.